초급반 1주차

이O재 @ AlKor, CYDF

목차

- 강사 소개
- 기초 환경 설정
- 프로그램 언어 소개
- 자료형
- 입출력
- 연습문제

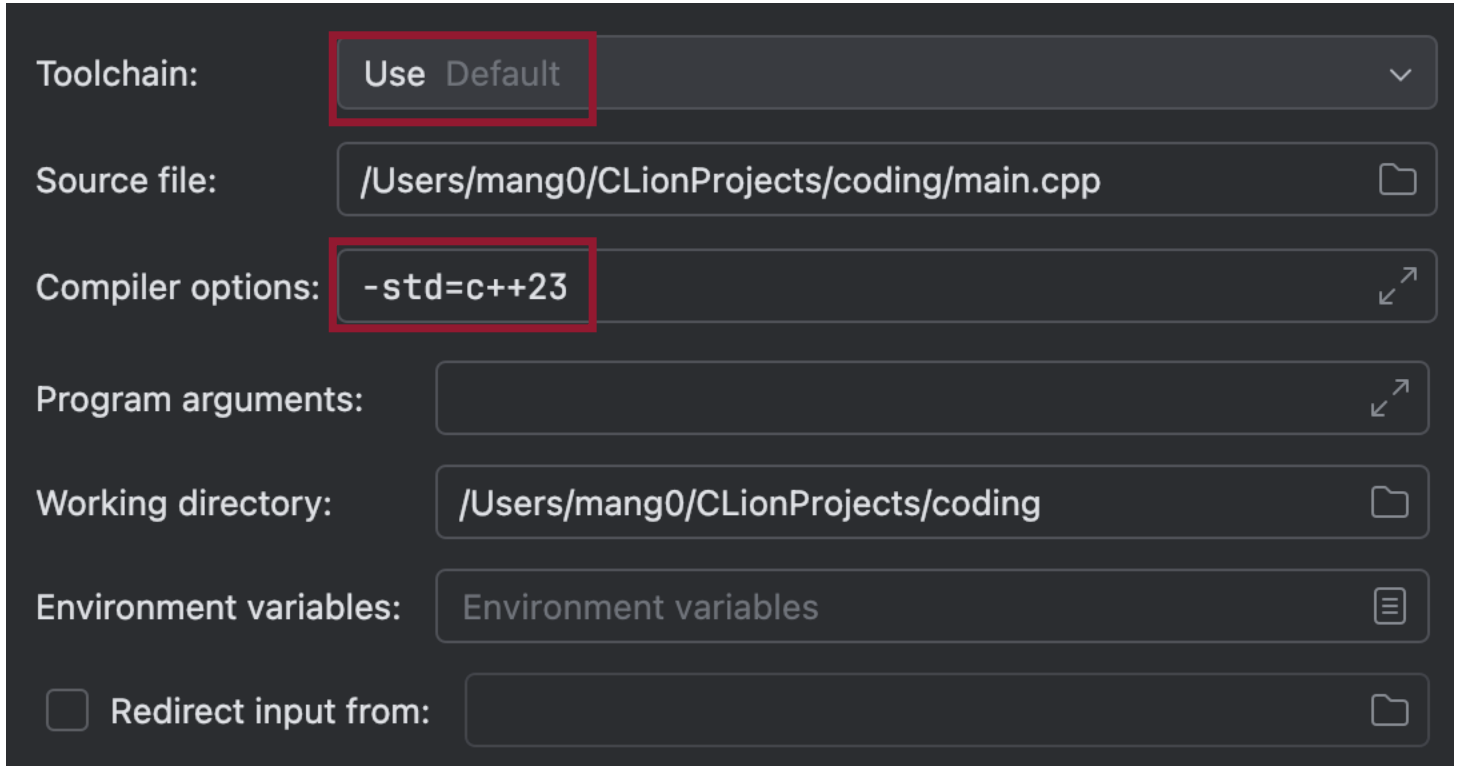
강사 소개

- 이름: 이O재
- 약력
  - 제14대 고려대학교 정보보호학부 학생회장
  - 2025년~2026년 정보보호학부 동아리 MatKor 회장
  - 2026년 사이버국방학과 동아리 AlKor 회장
  - 2026년 사이버국방학과 동아리 CyKor 부회장

  - 2025년 대통령과학장학생
  - 2025년 암호분석경진대회 최우수상
- 백준 닉네임: `leeminjae`
- 사이트: mang0.kr

기초 환경 설정

- Jetbrain CLION 사용 예정
  - 사용 이유: Mac/Window 모두 호환이 잘 되며, 초기 설정을 많이 하지 않아도 되어 좋음



프로그램 언어 소개 - C언어

- 역사
  - 1972년 데니스 리치(Dennis Ritchie)가 개발
  - 기존 언어인 **B언어**를 발전시켜 개발
  - UNIX 운영체제 개발에 사용함
  - 시스템 프로그래밍의 표준 언어로 자리잡음

- 특징
  - 중간 수준 언어(Assembly 언어와 거의 1대1 매칭)
  - 포인터 연산
  - 빠른 실행 속도

프로그램 언어 소개 - C++

- 역사
  - 1983년 비야네 스트롭스트룹(Bjarne Stroustrup)이 개발
  - C언어를 확장하여 객체지향 언어로 만듦

- 특징
  - C 기반이라 C 코드 대부분 호환
  - 객체지향 프로그래밍 지원(Class, 상속 등)
  - STL(Standard Template Library) 제공

- 수업시간에는 주로 C++를 다룰 예정

자료형

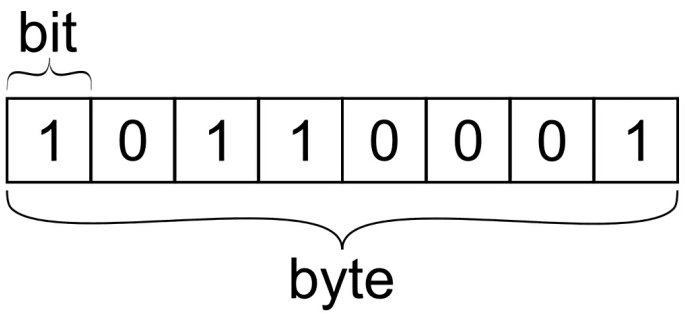
- 어떤 종류의 데이터를 저장할 것인지 컴퓨터에게 **사전**에 안내해주는 것

- 왜 사전에 안내 해주어야 하나?
  - 컴퓨터는 효율적인 동작을 위해 자료형 별로 사용할 크기가 다름
    e.g.) `int`: 4바이트, `char`: 1바이트, `long`: 8바이트 등
  - 컴퓨터가 미리 공간을 마련해두기 위해 필요함

- 자료형 종류
  - 문자 관련: `char`
  - 정수 관련: `char`, `int`, `long`, `long long`
  - 실수 관련: `float`, `double`, `long double`
  - 논리 관련: `bool`

자료형

- 바이트(byte)
  - 데이터를 저장하는 단위

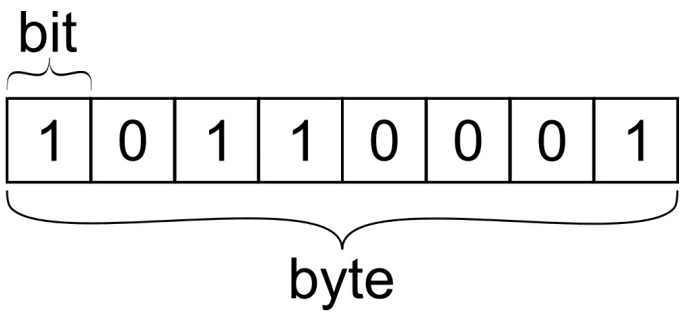


https://velog.io/@ichbinmin2/TIL.-1-바이트가-8비트인-이유

- Quiz. char는 몇 가지 종류의 숫자를 저장할 수 있을까?
  (Hint. char는 1바이트 자료형)

자료형

- 바이트(byte)
  - 데이터를 저장하는 단위



https://velog.io/@ichbinmin2/TIL.-1-바이트가-8비트인-이유

- Quiz. char는 몇 가지 종류의 숫자를 저장할 수 있을까?
  (Hint. char는 1바이트 자료형)
  - 1 비트 당 2가지 숫자(0, 1)를 저장할 수 있고, 8 비트가 있으므로 2^8

자료형

- char: 1바이트 정수

- 문자 저장에 사용(ASCII)

```
char a = ‘A’;
```

- 숫자 저장에 사용

```
char a = 65;
```

- char는 문자 인코딩을 통해 문자로 해석될 뿐, 1바이트 정수이다.

자료형

- ASCII

10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
0	0x00	NUL	32	0x20	Space	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	TAB	41	0x29	)	73	0x49	I	105	0x69	i

https://sheepone.tistory.com/47

- 외에도 UTF-8 등 인코딩 존재

자료형

- `int` : 4바이트 자료형
- `long` : 4 / 8바이트 자료형 (대부분 → 8, MSVC → 4)
- `long long` : 8바이트 자료형

```
#include <stdio.h>

int main() {
    printf("int: %zu\n", sizeof(int));
    printf("long: %zu\n", sizeof(long));
    printf("long long: %zu\n", sizeof(long long));
}
```

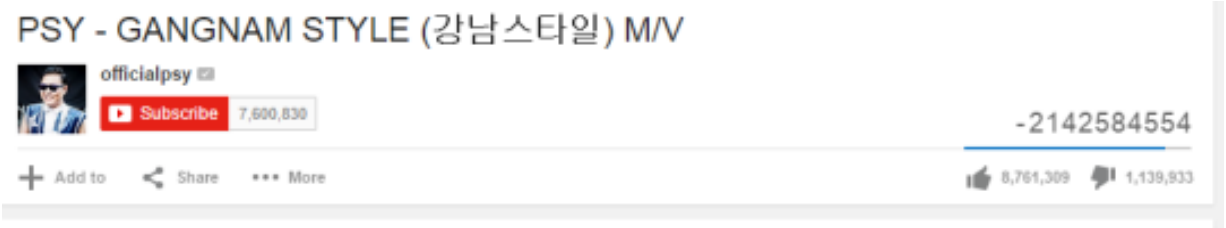
- 대부분 PS, 구현에서 `long long (int)`를 주로 사용

자료형

- 자료형별로 바이트 수가 다르기 때문에, 저장할 수 있는 범위가 다름

- `int`: $-2\,147\,483\,648\ (-2^{31}) \sim 2\,147\,483\,647\ (2^{31}-1)$
- `unsigned int`: $0 \sim 4\,294\,967\,295\ (2^{32}-1)$
- `long long`: $-2^{63} \sim 2^{63}-1$
- `unsigned long long`: $0 \sim 2^{64}-1$

- 왜 이런 숫자가 나오게 되었을까?



https://dalseobi.tistory.com/279

자료형

- 2진수 저장의 역사

- Sign Magnitude 방식
  - 맨 앞의 비트를 부호 비트로 생각하자

    +5 → 0000 0101

    -5 → 1000 0101

- 하지만 문제 발생
  - 0을 비효율적으로 저장함

    +0 → 0000 0000

    -0 → 1000 0000

자료형

- One's Complement (1의 보수) 방식
  - 음수를 모든 비트 반전으로 생각하자
    +5 → 0000 0101
    -5 → 1111 1010
- 하지만 문제 발생
  - 여전히 +0, -0 카운트
    +0 → 0000 0000
    -0 → 1111 1111
  - 덧셈 연산이 이상함
    +7 → 0000 0111
    -5 → 1111 1010

    +2 → 0000 0001

자료형

- Two's Complement (2의 보수) 방식
  - 음수를 비트 반전(1의 보수) + 1로 생각하자

    +5 → 0000 0101

    -5 → 1111 1011

- 모든 문제 해결 → 현재 컴퓨터 체계로 사용됨

- char 기준, -256(1000 0000)부터 255(0111 1111)까지 저장

자료형

- `float` : 4바이트 자료형
- `double` : 8바이트 자료형
- `long double` : 8 / **16**바이트 자료형

- 실수 자료형은 바이트가 많을 수록 범위와 **정밀도**에 영향을 미침

- 대부분 PS, 구현에서 `long double`을 주로 사용

자료형

- bool: 1바이트 자료형

- 저장하는 값
  - 0: false
  - 1: true

- 1비트만 있어도 될 거 같은데, 왜 1바이트나 필요하지?
  - 컴퓨터의 메모리 주소 지정 최소 바이트 수가 1바이트이기 때문
  - 나는 진짜 1비트만 쓰고 싶다 (메모리를 적게 쓰고 싶다)
    → std::vector<bool>, std::bitset 사용

입출력

- 입력: 사용자가 프로그램으로 데이터 전달
- 출력: 프로그램이 사용자로 데이터 전달

- 흐름: 사용자 → 입력 → 프로그램 처리 → 출력 → 사용자

- 사용자와 프로그램이 데이터(정보)를 주고받기 위해 사용

입출력 - C언어

- 입력: scanf
- 출력: printf

- 사용 예시

```
#include <stdio.h>

int main() {
    int a;
    scanf(“%d”, &a); // 입력
    printf(“%d”, a); // 출력
}
```

입출력 - C언어

- 형식 지정자
  - %d : 10진수 정수 출력
  - %f : 소수 출력 (소수점 아래 여섯째 자리까지)
  - %c : 문자 출력

- 형식 지정자의 다양한 표현
  - %2d : 10진수 정수를 2자리에 맞추어 출력
  - %02d : 10진수 정수를 2자리에 맞추어 출력하되, 앞자리는 0으로 출력
  - %.2f : 소수를 소수점 아래 셋째 자리에서 반올림하여 출력

입출력 - C언어

```c
#include <stdio.h>

int main() {
    int a = 10;
    printf("%d\n", a);
    printf("%3d\n", a);
    printf("%03d\n", a);

    printf("\n");

    float b = 3.14159265;
    printf("%f\n", b);
    printf("%.3f\n", b);

    return 0;
}
```

```
10
 10
010

3.141593
3.142
```

입출력

- escape 문자
  - 문자열 안에서 특별한 의미를 가지는 문자를 표현하는 방법

- 자주 사용하는 escape 문자
  - \n: 줄바꿈
  - \t: 탭
  - \\: \ 출력
  - \”: “ 출력
  - \’: ‘ 출력

입출력 - escape

```c
#include <stdio.h>

int main() {
    printf("Hello\nWorld");
    printf("\n\n");
    printf("Hello, my name is \"K\"");
}
```

```
Hello
World

Hello, my name is "K"
```

입출력 - C++

- 입력: std::cin
- 출력: std::cout

- 사용 예시

```
#include <iostream>

int main() {
    int a;
    std::cin >> a;  // 입력
    std::cout << a; // 출력
}
```

입출력 - C++

```cpp
#include <iomanip>
#include <iostream>

using namespace std;

int main() {
    float a = 3.141592;

    cout << a << '\n';
    cout << fixed << setprecision(3);
    cout << a << '\n';

    return 0;
}
```

```
3.14159
3.142
```

입출력 - 사칙연산

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int a = 7, b = 3;

    cout << a + b << '\n';
    cout << a - b << '\n';
    cout << a * b << '\n';
    cout << a / b << '\n';
    cout << a % b << '\n';

    return 0;
}
```

```
10
4
21
2
1
```

연습문제

- 고려대학교 2026-1R 초급 알고리즘 스터디(AlKor, ALPS, MatKor)

- 1주차 문제집

- 수업시간에 풀 문제: 2557, 1000, 10171
- 숙제: 1001, 10998, 10869, 18108, 10172

- Optional: 1008, 2588